

ปริญญานิพนธ์

การพัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล (Backend)

(Development of a Digital Signage Management System)

โดย

นายจักรภพ รามศักดิ์ 6501209

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตาม

หลักสูตรวิทยาศาสตรบัณฑิต (วท.บ.) สาขาวิชาวิทยาการคอมพิวเตอร์

วิทยาลัยนวัตกรรมดิจิทัลเทคโนโลยี

มหาวิทยาลัยรังสิต

ปีการศึกษา 2568

ใบรับรองปริญญานิพนธ์

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยนวัตกรรมดิจิทัลเทคโนโลยี

มหาวิทยาลัยรังสิต

เรื่อง การพัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล

โดย นายจักรภพ รามศักดิ์

ได้รับอนุญาตให้นับเป็นส่วนหนึ่งของการศึกษาตามหลักสูตร

ปริญญาวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์

.....

(อาจารย์ สุพานิช อังศิริกุล)

หัวหน้าสาขาวิชาวิทยาการคอมพิวเตอร์

วันที่ ..... เดือน ..... พ.ศ. .....

คณะกรรมการสอบปริญญานิพนธ์

ปิยวรรณ ทรัพย์แสนดี ..... ที่ปรึกษาและกรรมการ

(นางสาว ปิยวรรณ ทรัพย์แสนดี)

..... อาจารย์ที่ปรึกษาและกรรมการ

(ดร. อารีย์รัตน์ ส่งสกุลวัฒนา)

หัวข้อปริญญานิพนธ์	:	การพัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล
ผู้จัดทำ	:	นายจักรภพ รามศักดิ์ 6501209
อาจารย์ที่ปรึกษา	:	ดร. อารีย์รัตน์ ส่งสกุลวัฒนา
ที่ปรึกษาสถานประกอบการ	:	นางสาว ปิยวรรณ ทรัพย์แสนดี
หลักสูตรการศึกษา	:	วิทยาศาสตรบัณฑิต (วท.บ.)
สาขาวิชา	:	วิทยาการคอมพิวเตอร์
คณะ	:	วิทยาลัยนวัตกรรมดิจิทัลเทคโนโลยี
ปีการศึกษา	:	2568

บทคัดย่อ

จากการปฏิบัติงานสหกิจศึกษา ณ บริษัท บลูวินด์ โซลูชั่น จำกัด ซึ่งเป็นบริษัทผู้ให้บริการออกแบบและพัฒนาซอฟต์แวร์ ผู้จัดทำในฐานะนักพัฒนาระบบส่วนหลัง (Back-End Developer) ได้มีส่วนร่วมในการพัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล (Digital Signage Management System) ซึ่งพัฒนาขึ้นเพื่อแก้ปัญหาการบริหารจัดการจอประชาสัมพันธ์หลายจุดในองค์กรที่เดิมต้องอัปเดตเนื้อหาและควบคุมจอด้วยมือทีละเครื่อง โดยรวมการจัดการเนื้อหา การจัดผังการแสดงผล การกำหนดตารางเวลาออกอากาศ การควบคุมจอโทรทัศน์และจอ LED ระยะไกล ตลอดจนการประกาศเหตุฉุกเฉิน ไว้ในระบบกลางเพียงระบบเดียว

ในการนี้ บริษัทฯ ได้มอบหมายให้ผู้จัดทำรับผิดชอบการพัฒนาระบบส่วนหลัง ซึ่งประกอบด้วยการออกแบบและพัฒนา REST API ด้วย FastAPI การออกแบบฐานข้อมูล PostgreSQL ระบบสิทธิ์ผู้ใช้แบบ RBAC กระบวนการอนุมัติเนื้อหา การบันทึกประวัติการใช้งาน การควบคุมจอโทรทัศน์ระยะไกลผ่านโปรโตคอล HDMI-CEC และ RS-232 ตลอดจนชุดทดสอบอัตโนมัติ โดยระบบทั้งหมดทำงานในรูปแบบคอนเทนเนอร์ด้วย Docker ร่วมกับ Redis และ MinIO ระบบมีกระบวนการอนุมัติเนื้อหา การบันทึกประวัติการใช้งาน และการเฝ้าระวังสถานะอุปกรณ์ครบถ้วน ผลการพัฒนาได้ระบบที่ใช้งานได้ครบวงจรตั้งแต่การอัปโหลดเนื้อหาจนถึงการแสดงผลบนหน้าจอ โดยผ่านการทดสอบอัตโนมัติครบทั้ง 142 กรณี และการทดสอบแบบ End-to-End ครบทั้ง 10 ขั้นตอนบนฐานข้อมูล PostgreSQL จริง

คำสำคัญ: ป้ายประชาสัมพันธ์ดิจิทัล, การจัดการเนื้อหา, การควบคุมทีวีระยะไกล, HDMI-CEC, RS-232, FastAPI

Senior Project	:	Development of a Digital Signage Management System
By	:	Mr. Jakapob Ramsak ID 6501209
Advisor	:	Dr. Areerat Songsakulwattana
Company Supervisor	:	Ms. Piyawan Sapsandee
Curriculum	:	Bachelor of Science (B.Sc.)
Major	:	Computer Science
Faculty	:	College of Digital Innovation Technology
Academic Year	:	2026

ABSTRACT

During the cooperative education internship at Bluewind Solution Co., Ltd., a software design and development company, the author served as a Back-End Developer and contributed to the development of the Digital Signage Management System. The system was developed to address the challenge of managing multiple public-display screens within an organization, which previously required manual content updates and per-device screen control, by consolidating content management, layout composition, broadcast scheduling, remote control of televisions and LED displays, and emergency announcements into a single centralized platform.

In this capacity, the company assigned the author the responsibility of the development of the back-end system, covering the design and implementation of REST APIs with FastAPI, the PostgreSQL database design, role-based access control (RBAC), the content approval workflow, audit logging, remote television control via the HDMI-CEC and RS-232 protocols, and the automated test suite, with all services running in Docker containers alongside Redis and MinIO. The system provides a content approval workflow, audit logging, and device monitoring. The result is a fully functional system covering the workflow from content upload to on-screen display, passing all 142 automated test cases and all 10 steps of the end-to-end test on a live PostgreSQL database.

Keywords: Digital Signage, Content Management, Remote TV Control, HDMI-CEC, RS-232, FastAPI

กิตติกรรมประกาศ

ปริญญานิพนธ์ฉบับนี้สำเร็จลุล่วงได้ด้วยความกรุณาและการสนับสนุนจากหลายฝ่าย ผู้จัดทำขอขอบพระคุณ บริษัท บลูวินด์ โซลูชั่น จำกัด ที่ให้โอกาสเข้าปฏิบัติงานสหกิจศึกษาและเรียนรู้จากระบบงานจริง ขอขอบพระคุณ คุณปิยวรรณ ทรัพย์แสนดี ที่ปรึกษาสถานประกอบการ ที่ให้คำแนะนำด้านเทคนิคและการทำงานตลอดระยะเวลาการปฏิบัติงาน และขอขอบพระคุณ ดร. อารีย์รัตน์ ส่งสกุลวัฒนา อาจารย์ที่ปรึกษา ที่ให้คำปรึกษาด้านวิชาการและการจัดทำปริญญานิพนธ์ฉบับนี้

ขอขอบคุณ นางสาวพรชนก ยมรัตน์ เพื่อนร่วมโครงงานผู้รับผิดชอบส่วนติดต่อผู้ใช้ (Frontend) ที่ร่วมคิด ร่วมพัฒนา และร่วมแก้ปัญหาด้วยกันมาตลอดการดำเนินโครงงาน ขอขอบคุณคณาจารย์ทุกท่านที่ประสิทธิ์ประสาทวิชาความรู้ ตลอดจนครอบครัวและเพื่อน ๆ ที่ให้กำลังใจและการสนับสนุนด้วยดีเสมอมา คุณประโยชน์ใดที่เกิดจากปริญญานิพนธ์ฉบับนี้ ผู้จัดทำขอมอบแด่ผู้มีพระคุณทุกท่าน

ผู้จัดทำ

สารบัญ

หน้า

บทคัดย่อภาษาไทย ..... **ก**

บทคัดย่อภาษาอังกฤษ ..... **ข**

กิตติกรรมประกาศ ..... **ค**

สารบัญ ..... **ง**

สารบัญตาราง ..... **ช**

สารบัญรูปภาพ ..... **ซ**

บทที่ 1 บทนำ **1**

1.1 ความเป็นมาและความสำคัญของโครงงาน 1

1.2 วัตถุประสงค์ของโครงงาน 1

1.3 ขอบเขตของโครงงาน 2

1.3.1 ขอบเขตด้านปริมาณการใช้งาน 2

1.3.2 แพลตฟอร์มของจอแสดงผลที่รองรับ 2

1.3.3 การควบคุมจอโทรทัศน์และอุปกรณ์แสดงผล 2

1.3.4 ชนิดของเนื้อหาที่รองรับ 2

1.3.5 การติดตั้งและการนำไปใช้งาน 2

1.3.6 ข้อจำกัดของโครงงาน 2

1.4 ประโยชน์ที่คาดว่าจะได้รับ 3

1.5 หน้าที่ความรับผิดชอบของผู้จัดทำ 3

บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง **5**

2.1 ระบบป้ายประชาสัมพันธ์ดิจิทัล (Digital Signage) 5

2.2 FastAPI และภาษา Python 5

2.3 PostgreSQL, SQLAlchemy และ Alembic 6

2.4 Next.js, TypeScript และ Tailwind CSS 6

2.5 Redis และ Arq 6

2.6 MinIO 7

2.7 Docker และการทำคอนเทนเนอร์ (Containerization) 7
2.8 HDMI-CEC และ RS-232 8
2.9 ความปลอดภัยของระบบ (Security) 8

บทที่ 3 การวิเคราะห์และออกแบบระบบ 10
3.1 การวิเคราะห์ความต้องการของระบบ 10
3.2 เครื่องมือที่ใช้ในการพัฒนา 11
3.2.1 ฮาร์ดแวร์ (Hardware) 11
3.2.2 ซอฟต์แวร์ (Software) 11
3.3 การออกแบบระบบ 11
3.3.1 สถาปัตยกรรมระบบ (System Architecture) 11
3.3.2 การออกแบบฐานข้อมูล (ER Diagram) 12
3.3.3 แนวคิดลำดับชั้นเนื้อหาแบบ 5 ระดับ (5-Tier Content Hierarchy) 13
3.3.4 การออกแบบส่วนติดต่อโปรแกรม (API) 13
3.4 วิธีการพัฒนา 14
3.5 การทดสอบระบบ 14
3.5.1 การทดสอบอัตโนมัติ (Automated Test) 14
3.5.2 การทดสอบแบบ End-to-End (Smoke Test) 14
3.5.3 การทดสอบการควบคุมจอโทรทัศน์ด้วยตัวจำลอง (Emulator) 15

บทที่ 4 ผลการดำเนินงาน 16
4.1 ภาพรวมระบบที่พัฒนา 16
4.2 โมดูลและความสามารถหลัก 16
4.3 หน้าจอส่วนจัดการสำหรับผู้ดูแล (Admin UI) 17
4.4 ส่วนแสดงผล (Player) 24

บทที่ 5 สรุปผลและข้อเสนอแนะ 26
5.1 สรุปผลการดำเนินงาน 26
5.2 ปัญหาที่พบและแนวทางแก้ไข 26
5.3 ข้อเสนอแนะและการพัฒนาต่อยอด 26

5.4 สิ่งที่ได้เรียนรู้จากการฝึกงาน 27

บรรณานุกรม **28**

สารบัญตาราง

หน้า

ตารางที่

1.1 การแบ่งหน้าที่ความรับผิดชอบของผู้จัดทำ 4

3.1 แผนการดำเนินงานโครงงาน (Gantt Chart) หน่วยเป็นสัปดาห์ 11

4.1 โมดูลและความสามารถหลักของระบบ 17

5.1 ปัญหาที่พบและแนวทางแก้ไข 26

สารบัญรูปภาพ

หน้า

รูปที่

2.1	องค์ประกอบของระบบป้ายประชาสัมพันธ์ดิจิทัล	5
2.2	FastAPI	6
2.3	PostgreSQL	6
2.4	Next.js	6
2.5	Redis	7
2.6	MinIO	7
2.7	Docker	8
2.8	การควบคุมจอโทรทัศน์ผ่าน HDMI-CEC และ RS-232	8
2.9	มาตรการความปลอดภัยของระบบ	9
3.1	สถาปัตยกรรมของระบบจัดการจอประชาสัมพันธ์ดิจิทัล	12
3.2	แผนภาพความสัมพันธ์ของข้อมูล (ER Diagram)	13
4.1	หน้าเข้าสู่ระบบ (Login)	18
4.2	หน้าแดชบอร์ดสรุปสถานะระบบ (Dashboard)	18
4.3	หน้าจัดการสื่อและการอนุมัติ (Contents)	19
4.4	หน้าจัดผังการแสดงผล (Presentations)	20
4.5	หน้าจัดเพลย์ลิสต์ลำดับการเล่น (Playlists)	20
4.6	หน้าจัดตารางเวลาการแสดงผล (Schedules)	21
4.7	หน้าจัดการอุปกรณ์จอแสดงผล (Devices)	22
4.8	หน้าควบคุมจอโทรทัศน์ (TV Control)	22
4.9	หน้าจัดการประกาศฉุกเฉิน (Emergency)	23
4.10	หน้าบันทึกการใช้งานระบบ (Audit log)	23
4.11	ตัวอย่างการแสดงผลบนจอผ่าน Web Player	24
4.12	การแสดง overlay ประกาศฉุกเฉินบนจอแสดงผล	25

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของโครงงาน

ในปัจจุบัน องค์กรขนาดใหญ่ เช่น โรงพยาบาล สถานศึกษา และหน่วยงานราชการ นิยมใช้จอแสดงผลเพื่อประชาสัมพันธ์ข้อมูลข่าวสารแก่ผู้มาใช้บริการตามจุดต่าง ๆ ภายในอาคาร ระบบลักษณะนี้เรียกว่า "ระบบป้ายประชาสัมพันธ์ดิจิทัล" (Digital Signage) ซึ่งช่วยให้การสื่อสารข้อมูลทำได้รวดเร็วและปรับเปลี่ยนได้ตลอดเวลา อย่างไรก็ตาม เมื่อจำนวนจอเพิ่มมากขึ้นและกระจายอยู่หลายจุด การบริหารจัดการเนื้อหาและการควบคุมจอแต่ละเครื่องด้วยวิธีเดิม เช่น การนำไฟล์ใส่อุปกรณ์ USB ไปเสียบที่จอทีละเครื่อง หรือการเดินไปเปิด-ปิดจอด้วยมือ ก่อให้เกิดปัญหาหลายประการ ได้แก่ ความล่าช้าในการอัปเดตเนื้อหา ความไม่สอดคล้องของข้อมูลระหว่างจอ ความยากในการกำหนดตารางการแสดงผล และการขาดช่องทางในการประกาศข้อมูลเร่งด่วนหรือเหตุฉุกเฉินได้ทันท่วงที

จากปัญหาดังกล่าว ผู้จัดทำจึงได้พัฒนา "ระบบจัดการจอประชาสัมพันธ์ดิจิทัล (Digital Signage Management System)" ขึ้น เพื่อรวมการจัดการเนื้อหา การจัดผังการแสดงผล (layout) การกำหนดตารางเวลาออกอากาศ (schedule) การควบคุมจอโทรทัศน์ระยะไกล ตลอดจนการประกาศเหตุฉุกเฉิน ไว้ในระบบกลางเพียงระบบเดียว ผู้ดูแลสามารถสั่งงานผ่านเว็บแอปพลิเคชันจากศูนย์กลางได้ โดยไม่ต้องเดินไปจัดการที่หน้าจอแต่ละเครื่อง ช่วยลดทั้งเวลาและความผิดพลาดในการดำเนินงาน

ระบบออกแบบให้ทำงานภายในเครือข่ายภายในองค์กร (LAN) รองรับการเล่นเนื้อหาแบบ offline-friendly ผ่านไฟล์ manifest เพื่อให้จอยังแสดงผลต่อได้แม้เครือข่ายขัดข้องชั่วคราว มีระบบสิทธิ์ผู้ใช้ (RBAC) และกระบวนการอนุมัติเนื้อหาเพื่อความปลอดภัยและความถูกต้องของข้อมูล รวมถึงการบันทึกประวัติการใช้งาน (audit log) และการเฝ้าระวังสถานะอุปกรณ์ นอกจากนี้ยังพัฒนาด้วยเทคโนโลยี Open Source ทั้งหมด และติดตั้งด้วย Docker เพื่อให้ปรับใช้และดูแลรักษาได้ง่าย ลดต้นทุนค่าลิขสิทธิ์ซอฟต์แวร์

1.2 วัตถุประสงค์ของโครงงาน

1. เพื่อพัฒนาระบบจัดการเนื้อหาและตารางการแสดงผลของจอประชาสัมพันธ์จากศูนย์กลาง

2. เพื่อรองรับการควบคุมจอโทรทัศน์ระยะไกล (เปิด/ปิด/เปลี่ยนช่องสัญญาณเข้า) ผ่านโปรโตคอล HDMI-CEC และ RS-232 ได้หลายยี่ห้อ
3. เพื่อจัดให้มีระบบสิทธิ์ผู้ใช้ (RBAC) และกระบวนการอนุมัติเนื้อหาและตารางก่อนเผยแพร่
4. เพื่อรองรับการประกาศเหตุฉุกเฉินที่สามารถแทรก (override) ตารางการแสดงผลปกติได้ทันที
5. เพื่อเฝ้าระวังสถานะของอุปกรณ์และบันทึกประวัติการกระทำ (audit log) ทุกรายการ
6. เพื่อพัฒนาระบบด้วยเทคโนโลยี Open Source และสามารถติดตั้งใช้งานด้วย Docker ได้

1.3 ขอบเขตของโครงงาน

1.3.1 ขอบเขตด้านปริมาณการใช้งาน

รองรับการบริหารจัดการจอประชาสัมพันธ์ระดับ 10–20 จุด (ขนาด Proof of Concept) และสามารถขยายจำนวนได้ผ่านการจัดกลุ่มจอ (player group)

1.3.2 แพลตฟอร์มของจอแสดงผลที่รองรับ

รองรับ Android TV, แท็บเล็ต Android, Raspberry Pi (โหมด kiosk), เครื่องคอมพิวเตอร์ Windows และเว็บเบราว์เซอร์ (PWA)

1.3.3 การควบคุมจอโทรทัศน์และอุปกรณ์แสดงผล

รองรับการสั่งงานทีวีผ่าน HDMI-CEC และ RS-232 ได้หลายยี่ห้อ (เช่น Samsung MDC, LG, Sony) รวมถึงการควบคุมจอ LED ผ่านโมดูลควบคุม LED

1.3.4 ชนิดของเนื้อหาที่รองรับ

รองรับเนื้อหาประเภทรูปภาพ (image) วิดีโอ (video) หน้าเว็บ (web) สตรีม (stream) และข้อความ (text)

1.3.5 การติดตั้งและการนำไปใช้งาน

ออกแบบให้ติดตั้งแบบ on-premise บนระบบปฏิบัติการ Linux (Ubuntu 22.04) ด้วย Docker Compose

1.3.6 ข้อจำกัดของโครงงาน

เป็นการพัฒนาระบบใหม่ (greenfield) จึงไม่รวมการย้ายข้อมูลจากระบบเดิม (data migration)

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ลดเวลาและข้อผิดพลาดในการอัปเดตเนื้อหา เนื่องจากบริหารจัดการได้จากศูนย์กลาง
2. ลดภาระการเดินไปควบคุมจอแต่ละเครื่อง ด้วยการสั่งเปิด/ปิด/เปลี่ยนสัญญาณจากระยะไกล
3. เพิ่มความปลอดภัยและความถูกต้องของข้อมูล ด้วยระบบสิทธิ์ผู้ใช้และกระบวนการอนุมัติ
4. สามารถสื่อสารข้อมูลเร่งด่วนหรือเหตุฉุกเฉินไปยังทุกจอได้ทันที
5. ลดต้นทุนค่าลิขสิทธิ์ซอฟต์แวร์ ด้วยการใช้เทคโนโลยี Open Source
6. ได้ระบบต้นแบบที่สามารถนำไปขยายผลและต่อยอดในองค์กรจริงได้

1.5 หน้าที่ความรับผิดชอบของผู้จัดทำ

โครงงานนี้ดำเนินการร่วมกันโดยผู้จัดทำ 2 คน คือ นายจักรภพ รามศักดิ์ และ นางสาวพรชนก ยมรัตน์ โดยแบ่งความรับผิดชอบตามลักษณะงานออกเป็นส่วนระบบส่วนหลัง (Backend) และส่วนติดต่อผู้ใช้ (Frontend) ส่วนการออกแบบภาพรวมของระบบดำเนินการร่วมกัน ดังตารางที่ 1.1

ตารางที่ 1.1 การแบ่งหน้าที่ความรับผิดชอบของผู้จัดทำ

ส่วนงาน	**ผู้รับผิดชอบ**
ออกแบบสถาปัตยกรรมระบบ ฐานข้อมูล และรูปแบบการเชื่อมต่อ (API)	ร่วมกันทั้งสองคน
พัฒนา REST API และตรรกะทางธุรกิจฝั่งเซิร์ฟเวอร์ (FastAPI)	นายจักรภพ รามศักดิ์
ระบบสิทธิ์ผู้ใช้ (RBAC) กระบวนการอนุมัติเนื้อหา และ audit log	นายจักรภพ รามศักดิ์
การควบคุมจอโทรทัศน์/จอ LED ผ่าน HDMI-CEC และ RS-232 และงานเบื้องหลัง (Worker)	นายจักรภพ รามศักดิ์
ชุดทดสอบอัตโนมัติ 142 กรณี และระบบ CI	นายจักรภพ รามศักดิ์
ส่วนจัดการสำหรับผู้ดูแล (Admin UI) ด้วย Next.js	นางสาวพรชนก ยมรัตน์
ส่วนแสดงผล (Player) แบบ offline-friendly บนเว็บและ Android	นางสาวพรชนก ยมรัตน์
การออกแบบส่วนติดต่อผู้ใช้และประสบการณ์ใช้งาน (UX/UI)	นางสาวพรชนก ยมรัตน์
การทดสอบระบบรวม (End-to-End) และการจัดทำเอกสาร	ร่วมกันทั้งสองคน

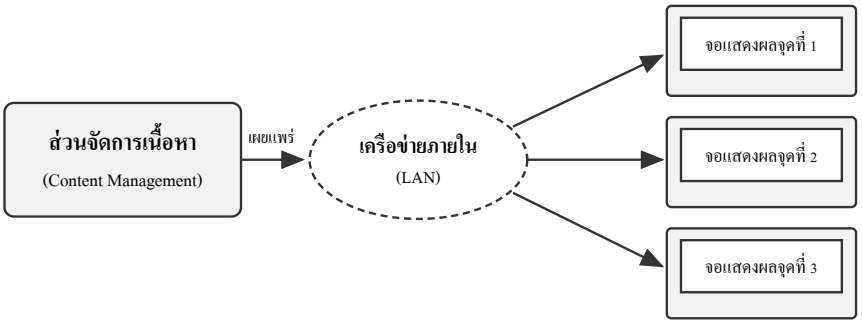
สำหรับเล่มฉบับนี้ ผู้จัดทำ (นายจักรภพ รามศักดิ์) รับผิดชอบส่วนระบบส่วนหลัง (Backend) เป็นหลัก เนื้อหาส่วนรายละเอียดการพัฒนาในบทที่ 3 จึงให้น้ำหนักกับการออกแบบและพัฒนาฝั่งเซิร์ฟเวอร์ โดยยังคงอธิบายภาพรวมของระบบทั้งหมดไว้เพื่อความสมบูรณ์ของเนื้อหา

บทที่ 2
ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

ในบทนี้กล่าวถึงทฤษฎีและเทคโนโลยีที่นำมาใช้ในการพัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล เพื่อให้เข้าใจหลักการทำงานของเครื่องมือแต่ละส่วนก่อนนำไปสู่การออกแบบและพัฒนาในบทต่อไป

2.1 ระบบป้ายประชาสัมพันธ์ดิจิทัล (Digital Signage)

ระบบป้ายประชาสัมพันธ์ดิจิทัล คือระบบที่ใช้จอแสดงผลอิเล็กทรอนิกส์ในการนำเสนอเนื้อหาดิจิทัล เช่น รูปภาพ วิดีโอ และข้อความ โดยสามารถควบคุมและปรับเปลี่ยนเนื้อหาจากศูนย์กลางผ่านเครือข่าย องค์ประกอบหลักประกอบด้วยส่วนจัดการเนื้อหา (Content Management) ส่วนแสดงผลที่ติดตั้งกับจอ (Player) และช่องทางสื่อสารระหว่างสองส่วนนี้ ทำให้องค์กรสามารถอัปเดตเนื้อหาและกำหนดตารางการแสดงผลล่วงหน้าได้อย่างมีประสิทธิภาพ



รูปที่ 2.1 องค์ประกอบของระบบป้ายประชาสัมพันธ์ดิจิทัล

2.2 FastAPI และภาษา Python

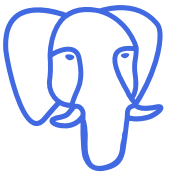
FastAPI เป็นเว็บเฟรมเวิร์กภาษา Python สำหรับสร้าง REST API ที่ทำงานแบบ asynchronous มีจุดเด่นด้านประสิทธิภาพสูง การตรวจสอบชนิดข้อมูลด้วย type hints และการสร้างเอกสาร API อัตโนมัติในรูปแบบ OpenAPI (Swagger) ในระบบนี้ใช้ FastAPI 0.115 ร่วมกับ Python 3.12 เป็นแกนหลักของฝั่ง backend สำหรับจัดการตรรกะทางธุรกิจและให้บริการ API แก่ส่วนอื่น ๆ



รูปที่ 2.2 FastAPI

2.3 PostgreSQL, SQLAlchemy และ Alembic

PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (RDBMS) แบบ open source ที่มีความเสถียรและรองรับข้อมูลปริมาณมาก ระบบนี้ใช้ PostgreSQL 16 เป็นฐานข้อมูลหลัก โดยเข้าถึงผ่าน SQLAlchemy 2.x ซึ่งเป็น Object-Relational Mapping (ORM) แบบ async และใช้ Alembic ในการจัดการการเปลี่ยนแปลงโครงสร้างฐานข้อมูล (migration) อย่างเป็นระบบ



รูปที่ 2.3 PostgreSQL

2.4 Next.js, TypeScript และ Tailwind CSS

Next.js เป็นเฟรมเวิร์กที่พัฒนาต่อยอดจาก React สำหรับสร้างเว็บแอปพลิเคชันที่รองรับทั้ง server-side rendering และ static generation ใช้ร่วมกับ TypeScript เพื่อเพิ่มความปลอดภัยด้านชนิดข้อมูล และ Tailwind CSS ซึ่งเป็น Utility-first CSS Framework สำหรับออกแบบหน้าตาให้ทันสมัยและรองรับการแสดงผลหลายขนาดหน้าจอ (responsive) ในระบบนี้ใช้สร้างหน้าจอส่วนจัดการสำหรับผู้ดูแล (Admin UI)



รูปที่ 2.4 Next.js

2.5 Redis และ Arq

Redis เป็นฐานข้อมูลแบบ in-memory ที่ทำงานได้รวดเร็ว ใช้สำหรับเก็บข้อมูลชั่วคราว (cache) และทำหน้าที่เป็นตัวกลางรับส่งข้อความ (message broker) ส่วน Arq เป็นตัวจัดคิวงาน (task queue) แบบ async ที่ทำงานร่วมกับ Redis ใช้รันงานเบื้องหลัง เช่น การหมดอายุของประกาศฉุกเฉินอัตโนมัติ การประมวลผลไฟล์สื่อ และคิวคำสั่งควบคุมทีวี



รูปที่ 2.5 Redis

2.6 MinIO

MinIO เป็นระบบจัดเก็บไฟล์แบบ object storage ที่เข้ากันได้กับ Amazon S3 API สามารถติดตั้งใช้งานภายในองค์กรเองได้ ในระบบนี้ใช้ MinIO สำหรับจัดเก็บไฟล์สื่อ เช่น รูปภาพและวิดีโอ ที่ผู้ใช้อัปโหลดเข้ามา



รูปที่ 2.6 MinIO

2.7 Docker และการทำคอนเทนเนอร์ (Containerization)

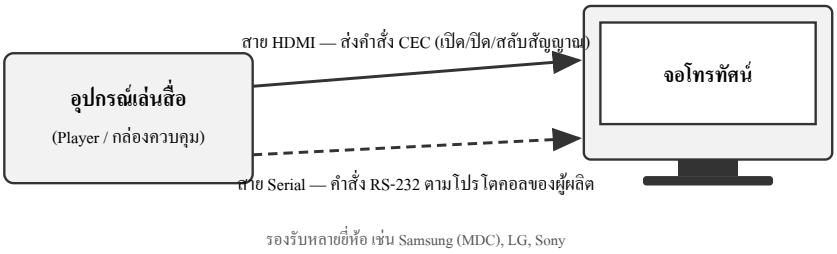
Docker เป็นเทคโนโลยีที่ช่วยบรรจุแอปพลิเคชันพร้อมสภาพแวดล้อมการทำงานไว้ในคอนเทนเนอร์ ทำให้ติดตั้งและทำงานได้เหมือนกันในทุกเครื่อง ระบบนี้ใช้ Docker Compose ในการจัดการบริการทั้งหมด (backend, worker, ฐานข้อมูล, Redis, MinIO, Admin UI) ให้ทำงานร่วมกัน ทำให้ติดตั้งระบบทั้งหมดได้ด้วยคำสั่งเดียว



รูปที่ 2.7 Docker

2.8 HDMI-CEC และ RS-232

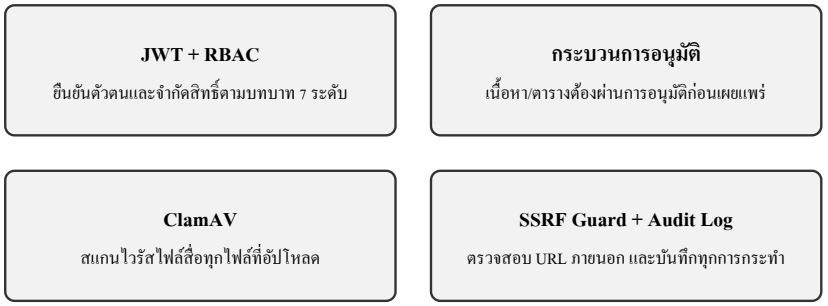
HDMI-CEC (Consumer Electronics Control) เป็นโปรโตคอลที่ส่งคำสั่งควบคุมอุปกรณ์ผ่านสาย HDMI เช่น การสั่งเปิด-ปิดทีวี ส่วน RS-232 เป็นมาตรฐานการสื่อสารแบบ serial ที่ทีวีระดับองค์กรหลายยี่ห้อรองรับสำหรับการควบคุมจากระยะไกล ระบบนี้ใช้ทั้งสองโปรโตคอลในการสั่งเปิด/ปิด/เปลี่ยนช่องสัญญาณเข้าของทีวีหลายยี่ห้อ (เช่น Samsung MDC, LG, Sony)



รูปที่ 2.8 การควบคุมจอโทรทัศน์ผ่าน HDMI-CEC และ RS-232

2.9 ความปลอดภัยของระบบ (Security)

ระบบให้ความสำคัญกับความปลอดภัยหลายระดับ ได้แก่ การยืนยันตัวตนด้วย JWT (JSON Web Token) ร่วมกับระบบสิทธิ์ตามบทบาท (Role-Based Access Control: RBAC), การสแกนไวรัสไฟล์ที่อัปโหลดด้วย ClamAV, และการป้องกันการโจมตีแบบ SSRF (Server-Side Request Forgery) ด้วยการตรวจสอบ URL ที่ผู้ใช้ส่งเข้ามาไม่ให้ชี้เข้าสู่เครือข่ายภายใน



รูปที่ 2.9 มาตรการความปลอดภัยของระบบ

บทที่ 3
การวิเคราะห์และออกแบบระบบ

ในบทนี้กล่าวถึงการวิเคราะห์ความต้องการของระบบ เครื่องมือที่ใช้ และการออกแบบระบบในด้านสถาปัตยกรรม ฐานข้อมูล และส่วนติดต่อโปรแกรม (API)

3.1 การวิเคราะห์ความต้องการของระบบ

ความต้องการเชิงหน้าที่ (Functional Requirements):

- จัดการเนื้อหาสื่อ (อัปโหลด/อนุมัติ/จัดหมวด)
- จัดผังการแสดงผลแบบหลายโซน (presentation/layout)
- กำหนดตารางการแสดงผลตามช่วงเวลาและกลุ่มจอ
- ควบคุมจอโทรทัศน์ระยะไกล (เปิด/ปิด/เปลี่ยนสัญญาณ)
- ประกาศเหตุฉุกเฉินที่แทรกตารางปกติได้ทันที
- จัดการผู้ใช้และสิทธิ์การเข้าถึง (RBAC)
- เฝ้าระวังสถานะอุปกรณ์และบันทึก audit log

ความต้องการที่ไม่ใช่เชิงหน้าที่ (Non-functional Requirements):

- ความปลอดภัย (การยืนยันตัวตน สิทธิ์ และการตรวจสอบย้อนหลัง)
- ความทนทานต่อเครือข่ายขัดข้อง (offline-friendly ผ่าน manifest)
- ความง่ายในการติดตั้งและดูแล (Docker)
- ใช้เทคโนโลยี Open Source

ระยะเวลาดำเนินงานรวมประมาณ 16 สัปดาห์ แบ่งงานหลักออกเป็น 7 ระยะ ดังตารางที่ 3.1

ตารางที่ 3.1 แผนการดำเนินงานโครงงาน (Gantt Chart) หน่วยเป็นสัปดาห์

งาน / สัปดาห์ที่	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1. ศึกษาระบบงานเดิมและรวบรวมความต้องการ	■	■														
2. ออกแบบสถาปัตยกรรมระบบและฐานข้อมูล		■	■	■												
3. พัฒนาระบบฝั่งเซิร์ฟเวอร์ (Backend API)				■	■	■	■	■	■							
4. พัฒนาส่วนจัดการสำหรับผู้ดูแล (Admin UI)						■	■	■	■	■	■					
5. พัฒนาส่วนแสดงผลและการควบคุมทีวี (Player)									■	■	■	■	■			
6. ทดสอบระบบและแก้ไขข้อบกพร่อง												■	■	■	■	
7. จัดทำเอกสารและเล่มรายงาน														■	■	■

3.2 เครื่องมือที่ใช้ในการพัฒนา

3.2.1 ฮาร์ดแวร์ (Hardware)

- เครื่องคอมพิวเตอร์สำหรับพัฒนาและทดสอบระบบ (Developer PC)
- อุปกรณ์แสดงผลที่ใช้ทดสอบ: เว็บเบราว์เซอร์ในโหมดเต็มจอ (kiosk) บนเครื่องคอมพิวเตอร์ ทำหน้าที่เป็นจอแสดงผลของระบบ
- การทดสอบการควบคุมจอโทรทัศน์: ใช้ตัวจำลอง (emulator) ที่ผู้จัดทำพัฒนาขึ้น ร่วมกับพอร์ตอนุกรมเสมือน (virtual serial port) แทนจอโทรทัศน์จริง เนื่องจากข้อจำกัดด้านอุปกรณ์ในช่วงพัฒนา (รายละเอียดในหัวข้อ 3.5.3)

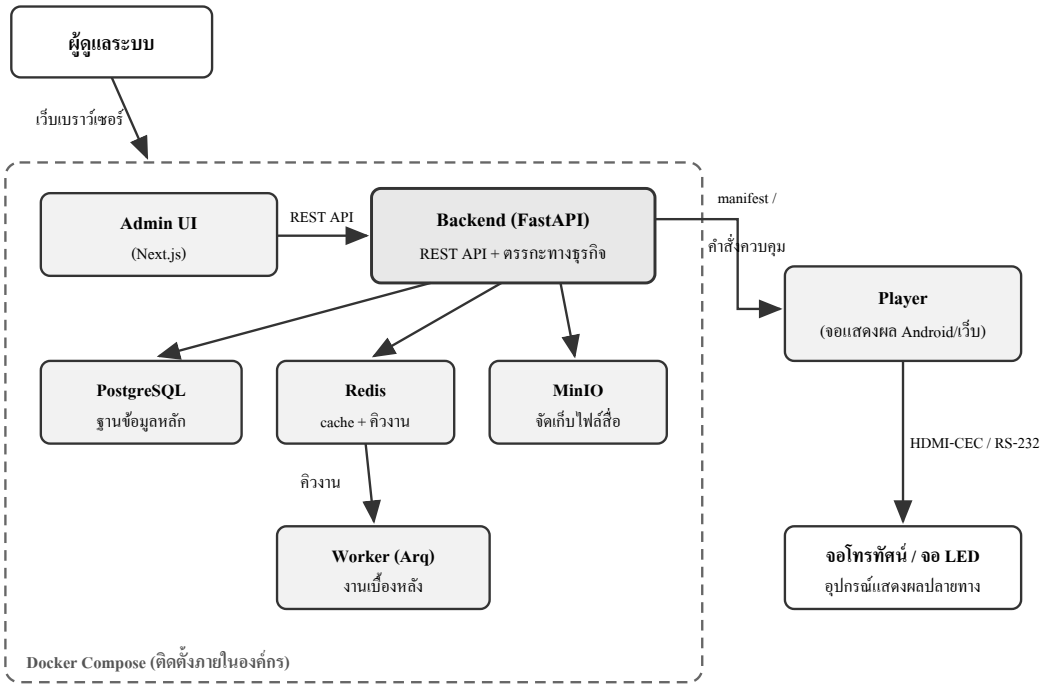
3.2.2 ซอฟต์แวร์ (Software)

- ภาษา/เฟรมเวิร์ก: Python 3.12 + FastAPI, Node.js + Next.js + TypeScript
- ฐานข้อมูลและบริการ: PostgreSQL 16, Redis 7, MinIO
- เครื่องมือพัฒนา: Visual Studio Code, Git/GitHub, Docker

3.3 การออกแบบระบบ

3.3.1 สถาปัตยกรรมระบบ (System Architecture)

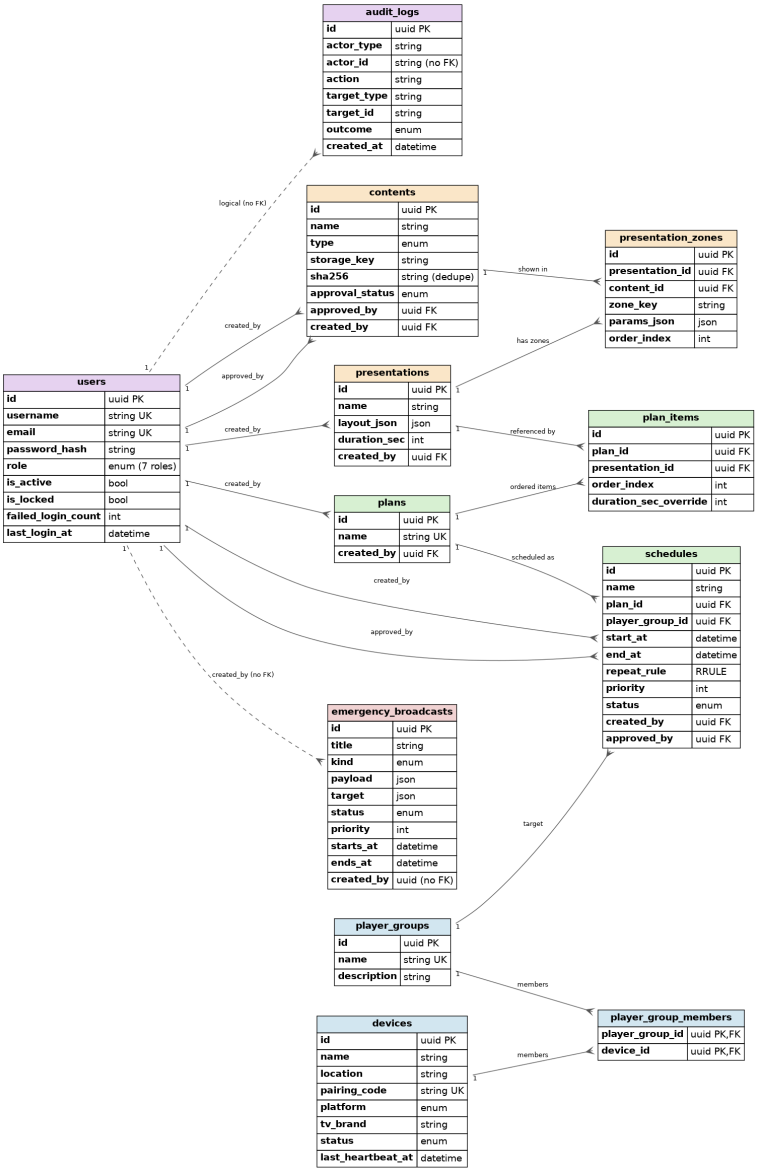
ระบบประกอบด้วยส่วนหลัก 6 ส่วนที่ทำงานร่วมกันใน Docker stack ได้แก่ (1) Backend (FastAPI) ให้บริการ REST API และตรรกะทางธุรกิจ (2) Worker (Arq) สำหรับงานเบื้องหลัง (3) PostgreSQL ฐานข้อมูลหลัก (4) Redis สำหรับ cache และคิวงาน (5) MinIO สำหรับจัดเก็บไฟล์สื่อ และ (6) Admin UI (Next.js) สำหรับผู้ดูแล โดยส่วนแสดงผล (Player) จะดึงไฟล์ manifest จาก backend แล้วเล่นเนื้อหาแบบ offline-friendly ดังรูปที่ 3.1



รูปที่ 3.1 สถาปัตยกรรมของระบบจัดการจอประชาสัมพันธ์ดิจิทัล

3.3.2 การออกแบบฐานข้อมูล (ER Diagram)

ฐานข้อมูลประกอบด้วยตารางประมาณ 13 ตาราง (entity 12 ตาราง + ตารางความสัมพันธ์แบบ M:N) แบ่งเป็น 5 กลุ่ม ได้แก่ Identity & Audit, Devices, Content & Composition, Playlists & Schedules และ Emergency ดังรูปที่ 3.2



รูปที่ 3.2 แผนภาพความสัมพันธ์ของข้อมูล (ER Diagram)

3.3.3 แนวคิดลำดับชั้นเนื้อหาแบบ 5 ระดับ (5-Tier Content Hierarchy)

ระบบออกแบบลำดับชั้นเนื้อหาเป็น 5 ระดับ คือ Content (สื่อดิบ) ⟶ Zone (โซนแสดงผล) ⟶ Presentation (ผังการจัดวาง) ⟶ Plan (ลำดับการเล่น) ⟶ Schedule (เวลาและกลุ่มจอ) แนวคิดนี้ทำให้สามารถนำเนื้อหากลับมาใช้ซ้ำได้ และการแก้ไขตารางครั้งเดียวมีผลกับทั้งกลุ่มจอ

3.3.4 การออกแบบส่วนติดต่อโปรแกรม (API)

API เวอร์ชัน v1 มีทั้งหมดประมาณ 66 endpoints จัดเป็น 13 กลุ่ม (auth, users, devices, groups, contents, presentations, plans, schedules, player, tv, emergency, led, reports) ออกแบบ

ตามแนวทาง REST ยืนยันตัวตนด้วย JWT และมีเอกสารอัตโนมัติแบบ OpenAPI (Swagger)

3.4 วิธีการพัฒนา

พัฒนาด้วยกระบวนการแบบวนซ้ำเป็นเฟส (iterative) ใช้ Git/GitHub ในการจัดการเวอร์ชันโค้ด มีระบบ Continuous Integration (GitHub Actions) สำหรับรันการทดสอบและตรวจสอบความปลอดภัยอัตโนมัติ และใช้ Docker เป็นสภาพแวดล้อมการพัฒนาที่สอดคล้องกับ production โดยผู้จัดทำทั้งสองคนแบ่งความรับผิดชอบตามตารางที่ 1.1 และตกลงรูปแบบการเชื่อมต่อ (API) ร่วมกันก่อนแยกกันพัฒนาแต่ละฝั่ง

ในส่วนความรับผิดชอบของผู้จัดทำ ได้พัฒนาระบบส่วนหลังทั้งหมดของโครงงาน ได้แก่ การออกแบบและพัฒนา REST API ประมาณ 66 endpoints ใน 13 กลุ่มด้วย FastAPI แบบ asynchronous การออกแบบโครงสร้างฐานข้อมูล PostgreSQL ทั้ง 13 ตาราง การจัดการ cache และคิวงานด้วย Redis ร่วมกับ Worker (Arq) สำหรับงานเบื้องหลัง การจัดเก็บไฟล์สื่อด้วย MinIO ระบบยืนยันตัวตนด้วย JWT และสิทธิ์ผู้ใช้แบบ RBAC 7 บทบาท กระบวนการอนุมัติเนื้อหา การบันทึก audit log ชั้นควบคุมจอโทรทัศน์หลายยี่ห้อผ่านโปรโตคอล HDMI-CEC และ RS-232 การป้องกัน SSRF และการสแกนไฟล์อัปโหลดด้วย ClamAV ตลอดจนชุดทดสอบอัตโนมัติ 142 กรณี และการตั้งค่าระบบ CI

3.5 การทดสอบระบบ

การทดสอบระบบแบ่งออกเป็น 3 ระดับ ได้แก่ การทดสอบอัตโนมัติระดับหน่วย การทดสอบแบบ End-to-End บนระบบที่ติดตั้งจริง และการทดสอบการควบคุมจอโทรทัศน์ด้วยตัวจำลอง

3.5.1 การทดสอบอัตโนมัติ (Automated Test)

ระบบใช้ pytest ร่วมกับ pytest-asyncio และ pytest-cov มีกรณีทดสอบ 142 กรณีใน 15 ไฟล์ ครอบคลุมการยืนยันตัวตน อุปกรณ์ กระบวนการจัดการเนื้อหา การควบคุมทีวี การประกาศฉุกเฉินและ audit, มิดเดิลแวร์ความปลอดภัย, รายงาน (reports), การควบคุม LED, การป้องกัน SSRF และงานเบื้องหลัง ผลการทดสอบผ่านครบทั้ง 142 กรณี นอกจากนี้ยังมีระบบ CI (GitHub Actions) ที่รันชุดทดสอบดังกล่าวโดยอัตโนมัติทุกครั้งที่มีการแก้ไขโค้ด เพื่อป้องกันข้อผิดพลาดย้อนกลับ (regression)

3.5.2 การทดสอบแบบ End-to-End (Smoke Test)

เป็นการทดสอบที่เดินครบทุกขั้นตอนการใช้งานจริงบนระบบที่ติดตั้งด้วย Docker และใช้ฐานข้อมูล PostgreSQL จริง (ไม่ใช่ฐานข้อมูลจำลอง) รวม 10 ขั้นตอน ได้แก่ (1) ตรวจสอบความพร้อมของบริการ (2) เข้าสู่ระบบและต่ออายุโทเคน (3) สร้างและจับคู่อุปกรณ์พร้อมส่งสัญญาณ heartbeat (4) เพิ่มเนื้อหาและอนุมัติ (5) สร้างผังการแสดงผล เพลย์ลิสต์ และตารางเวลา (6) ตรวจสอบไฟล์ manifest ที่ player ได้รับ (7) ประกาศเหตุฉุกเฉินและตรวจสอบการแทรกตารางปกติ (8) คิวคำสั่งควบคุมทีวี (9) การบันทึก audit log และ (10) การล้างข้อมูลทดสอบ ผลการทดสอบผ่านครบทั้ง 10 ขั้นตอน

3.5.3 การทดสอบการควบคุมจอโทรทัศน์ด้วยตัวจำลอง (Emulator)

เนื่องจากในช่วงพัฒนายังไม่มีจอโทรทัศน์ที่รองรับ RS-232 ให้ทดสอบ ผู้จัดทำจึงพัฒนาตัวจำลอง (emulator) ขึ้นเพื่อตรวจสอบความถูกต้องของคำสั่งควบคุมโดยไม่ต้องใช้อุปกรณ์จริง ประกอบด้วยตัวจำลองจอโทรทัศน์ที่ถอดรหัส (decode) ไบต์คำสั่งตามสเปกของแต่ละยี่ห้อ ตัวจำลองส่วนแสดงผล และตัวจำลองอุปกรณ์ HDMI-CEC วิธีทดสอบคือให้ระบบสร้างไบต์คำสั่งจริงแล้วส่งให้ตัวจำลองถอดรหัสกลับ เพื่อเปรียบเทียบว่าคำสั่งที่ได้ตรงกับที่สั่งหรือไม่

ผลการทดสอบพบว่าคำสั่งทั้งหมด 47 รูปแบบ ครอบคลุม 6 ยี่ห้อ (Samsung MDC, LG, Sony, Panasonic, NEC, Toshiba) และโหมด HDMI-CEC ถูกถอดรหัสได้ถูกต้องทุกคำสั่ง ไม่พบข้อผิดพลาด และเมื่อทดสอบส่งผ่านพอร์ตอนุกรมเสมือน (virtual serial port) ข้อมูลที่อ่านกลับมาตรงกับที่ส่งไปทุกไบต์ การทดสอบระดับนี้ยืนยันความถูกต้องของการเข้ารหัสคำสั่งตามโปรโตคอล ส่วนการทดสอบกับจอโทรทัศน์จริงยังคงเป็นงานที่ต้องดำเนินการต่อไป (ดูหัวข้อ 5.3)

บทที่ 4
ผลการดำเนินงาน

ในบทนี้นำเสนอผลการพัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล ทั้งในส่วนความสามารถของแต่ละโมดูล หน้าจอการใช้งานจริง และส่วนแสดงผล

4.1 ภาพรวมระบบที่พัฒนา

ระบบได้รับการพัฒนาจนสามารถใช้งานได้ครบวงจร ประกอบด้วยส่วน backend (REST API), ส่วนจัดการสำหรับผู้ดูแล (Admin UI) และส่วนแสดงผล (Player) ทั้งบนเว็บและ Android รองรับกระบวนการทำงานตั้งแต่การอัปโหลดเนื้อหา ⟶ จัดผังการแสดงผล ⟶ ตั้งตารางเวลา ⟶ แสดงผลบนจอ ⟶ การประกาศเหตุฉุกเฉิน

4.2 โมดูลและความสามารถหลัก

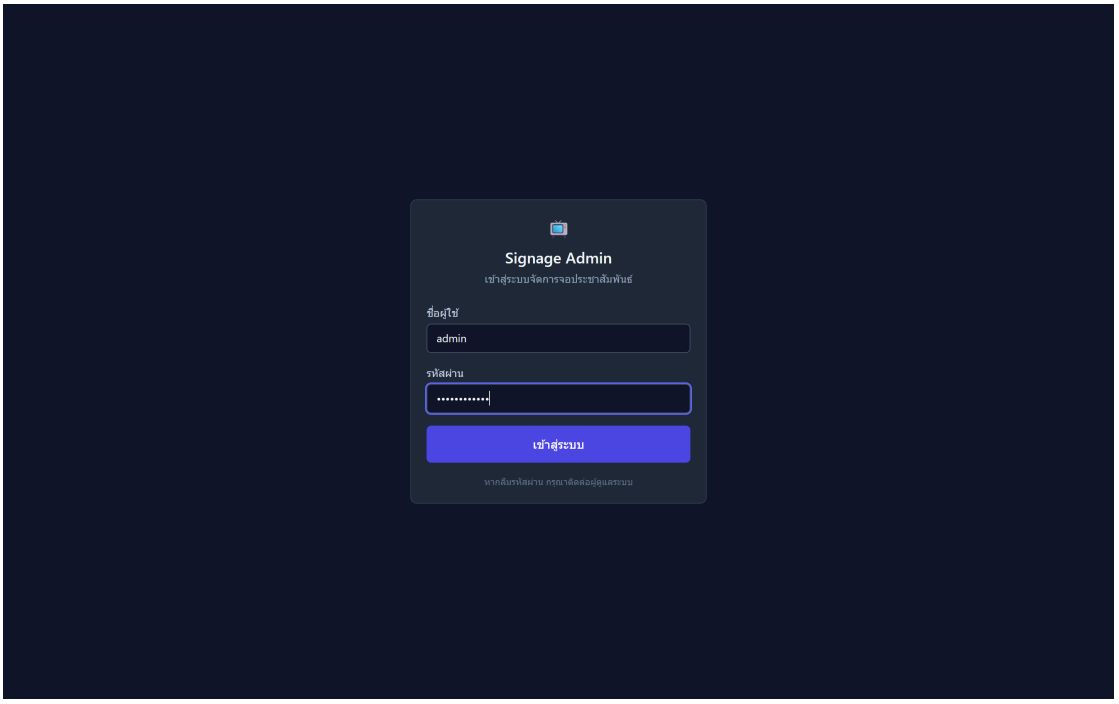
ระบบประกอบด้วยโมดูลการทำงานหลักและความสามารถของแต่ละส่วน สรุปได้ดังตารางที่ 4.1

ตารางที่ 4.1 โมดูลและความสามารถหลักของระบบ

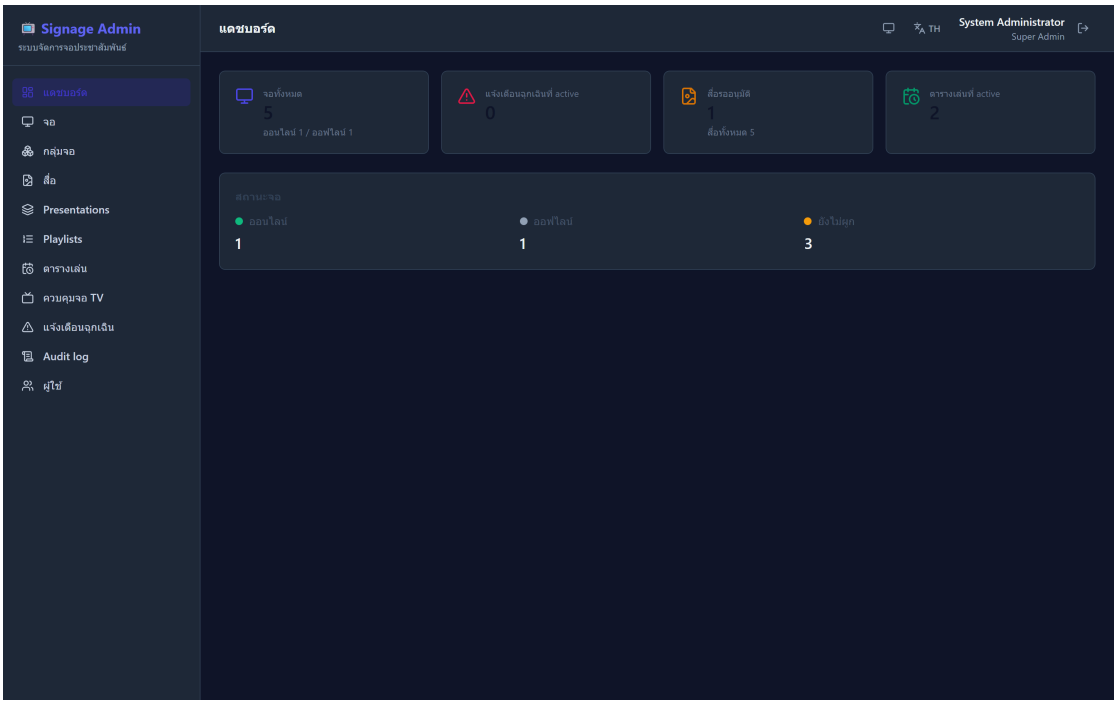
โมดูล	**ความสามารถ**
Auth & Users	ยืนยันตัวตนด้วย JWT + RBAC 7 บทบาท, ล็อกบัญชีเมื่อล็อกอินผิดซ้ำ, รองรับ cookie auth
Devices	จับคู่อุปกรณ์ด้วยรหัส, heartbeat, telemetry (CPU/อุณหภูมิ/uptime), จัดกลุ่มจอแบบ M:N
Contents	อัปโหลดสื่อ + ตรวจซ้ำด้วย sha256 + สร้าง thumbnail + กระบวนการอนุมัติ (draft⟶pending⟶approved/rejected)
Presentations	จัดผัง layout หลายโซน (composer) ใน Admin UI
Plans	จัดลำดับ presentation พร้อมกำหนดระยะเวลาแสดงผล
Schedules	ผูก plan กับกลุ่มจอ + กฎเวลาแบบ RRULE (iCal) + ช่วงเวลารายวัน + ลำดับความสำคัญ + การอนุมัติ
TV Control	สั่งเปิด/ปิด/เปลี่ยนสัญญาณผ่าน HDMI-CEC และ RS-232 หลายยี่ห้อ ผ่านคิวใน Redis
LED Control	ควบคุมจอ LED ผ่านโมดูลควบคุมเฉพาะ *(ฟีเจอร์เพิ่มเติม)*
Emergency	ประกาศฉุกเฉิน override ทุกตาราง + หมดอายุอัตโนมัติ + ต้องการการยืนยันรับทราบ (ack)
Reporting	บันทึกประวัติการเล่น (playback_logs) + สรุปรายงานและส่งออกข้อมูล *(ฟีเจอร์เพิ่มเติม)*
Monitoring	Prometheus metrics + readiness/liveness + audit log + Sentry + การสำรองข้อมูล
Player	แสดงผลหลายโซน + วนเล่นตาม plan + overlay ประกาศฉุกเฉิน + cache แบบ offline

4.3 หน้าจอส่วนจัดการสำหรับผู้ดูแล (Admin UI)

ส่วนจัดการประกอบด้วยหน้าจอหลักหลายหน้า ผู้ใช้เข้าสู่ระบบด้วยชื่อผู้ใช้และรหัสผ่าน (รูปที่ 4.1) จากนั้นระบบแสดงหน้าแดชบอร์ดสรุปสถานะภาพรวม ได้แก่ จำนวนจอทั้งหมดพร้อมสถานะออนไลน์ สื่อที่รออนุมัติ และตารางเล่นที่กำลังทำงาน (รูปที่ 4.2)



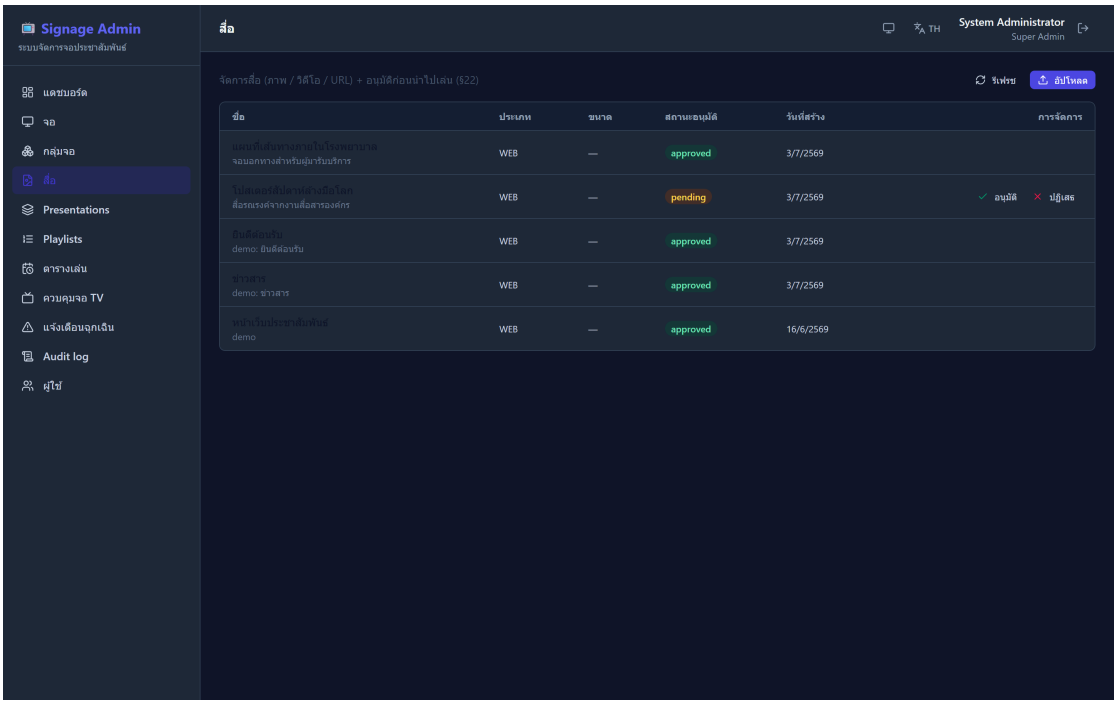
รูปที่ 4.1 หน้าเข้าสู่ระบบ (Login)



รูปที่ 4.2 หน้าแดชบอร์ดสรุปสถานะระบบ (Dashboard)

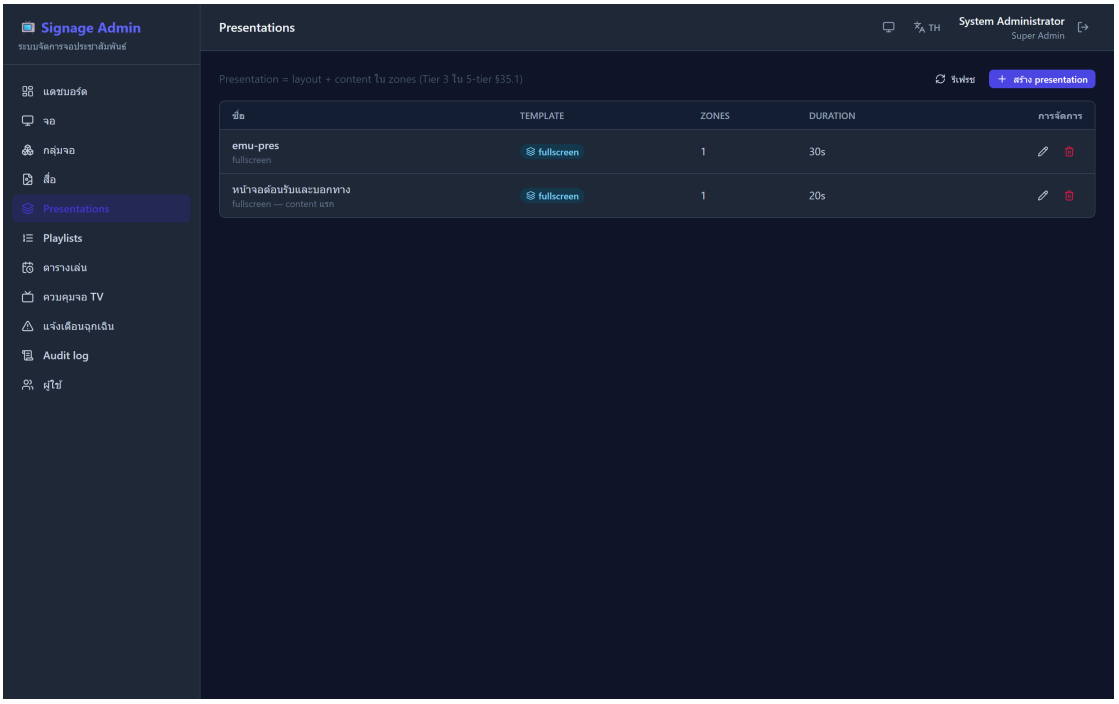
หน้าจัดการสื่อรองรับการเพิ่มสื่อทั้งแบบไฟล์และ URL โดยสื่อที่เพิ่มใหม่ต้องผ่านกระบวนการอนุมัติก่อนนำไปแสดงผลจริง ผู้มีสิทธิ์สามารถกดอนุมัติหรือปฏิเสธได้จากหน้า

รายการโดยตรง (รูปที่ 4.3)

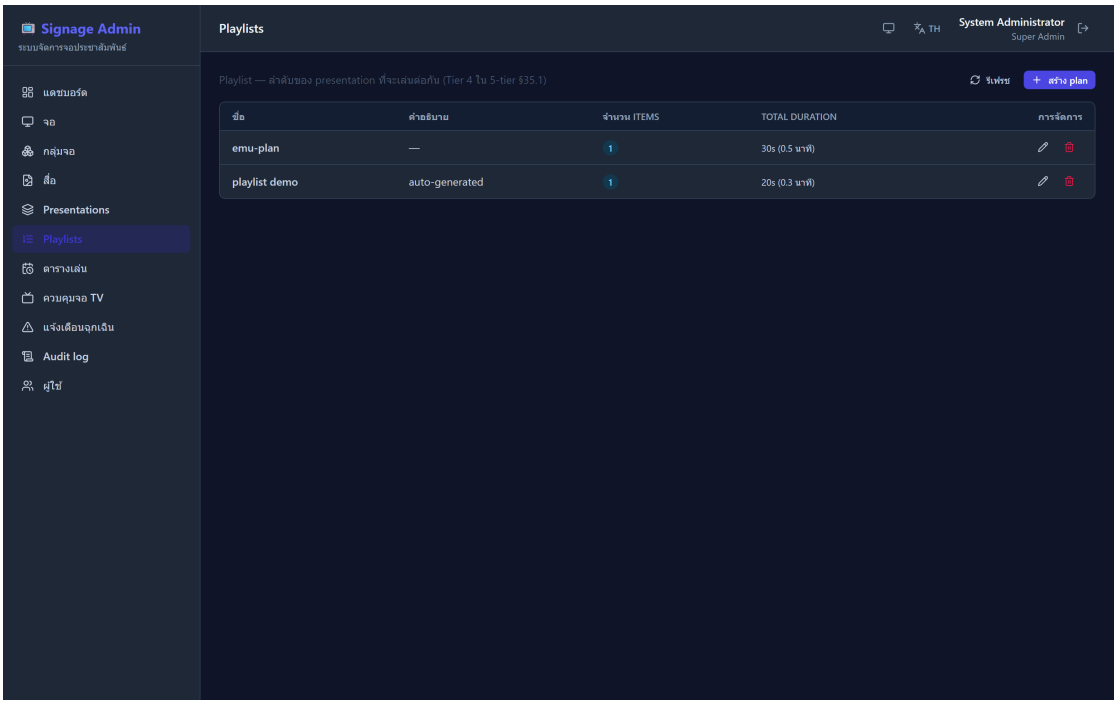


รูปที่ 4.3 หน้าจัดการสื่อและการอนุมัติ (Contents)

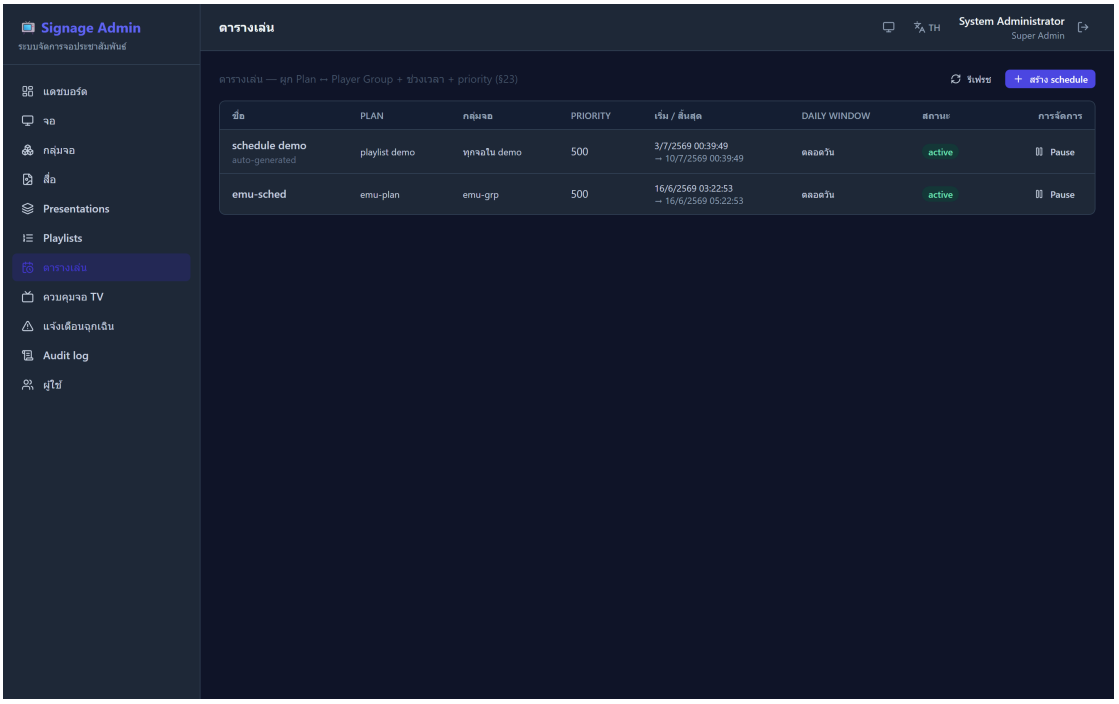
การจัดผังการแสดงผลทำผ่าน presentation ซึ่งกำหนด layout และเนื้อหาในแต่ละโซน จากนั้นนำ presentation มาเรียงลำดับเป็นเพลย์ลิสต์ และผูกกับกลุ่มจอผ่านตารางเวลาที่กำหนดช่วงเวลาและลำดับความสำคัญ (รูปที่ 4.4–4.6)



รูปที่ 4.4 หน้าจัดผังการแสดงผล (Presentations)

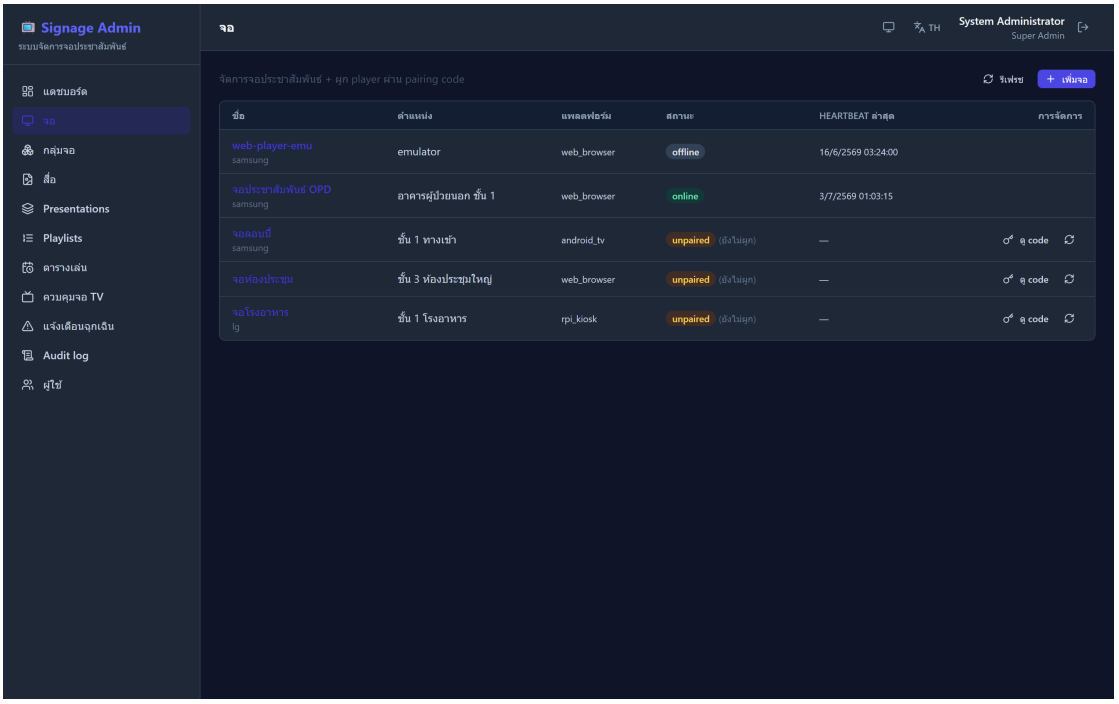


รูปที่ 4.5 หน้าจัดเพลย์ลิสต์ลำดับการเล่น (Playlists)

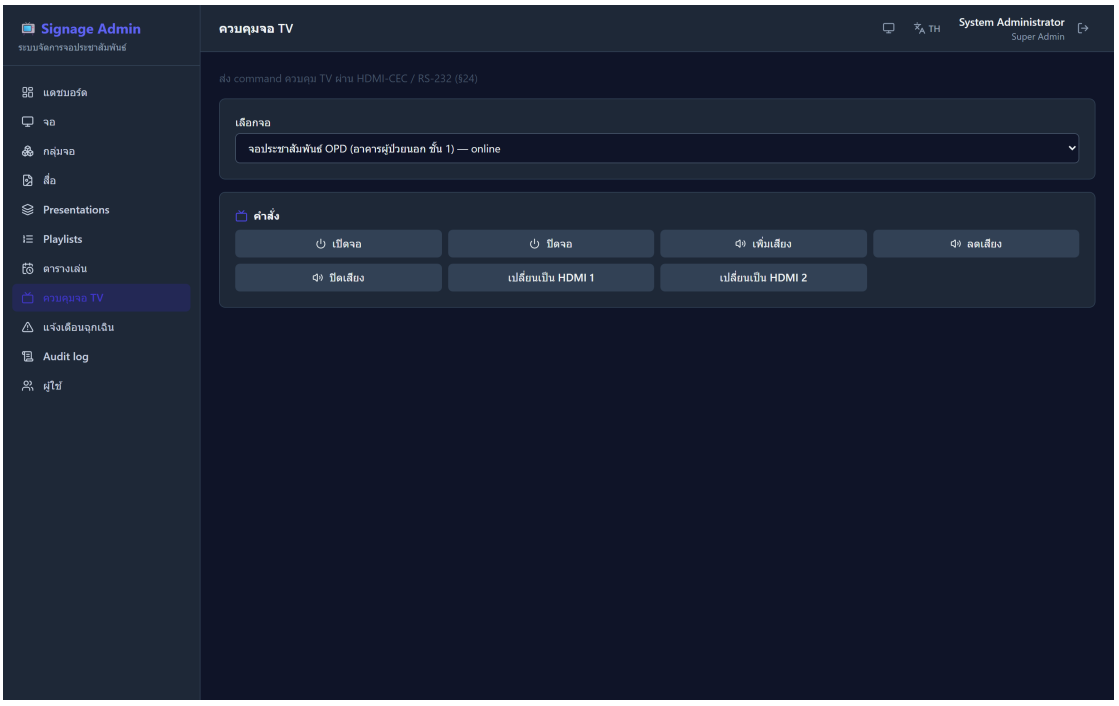


รูปที่ 4.6 หน้าจัดตารางเวลาการแสดงผล (Schedules)

หน้าจัดการอุปกรณ์แสดงรายการจอทั้งหมดพร้อมสถานะ (ออนไลน์ / ออฟไลน์ / ยังไม่ผูก) รหัสสำหรับจับคู่อุปกรณ์ และเวลา heartbeat ล่าสุดของแต่ละจอ (รูปที่ 4.7) ส่วนหน้าควบคุมจอโทรทัศน์ใช้ส่งคำสั่ง เปิด/ปิดจอ ปรับเสียง และเปลี่ยนช่องสัญญาณ HDMI ไปยังจอที่เลือกผ่าน HDMI-CEC หรือ RS-232 (รูปที่ 4.8)



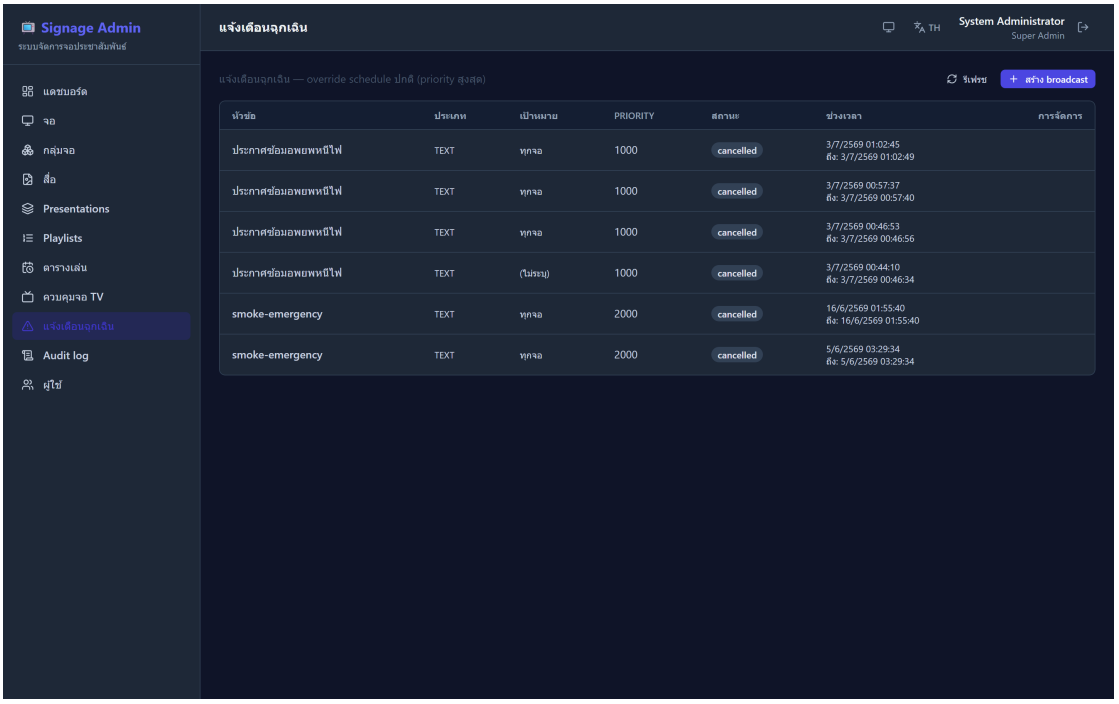
รูปที่ 4.7 หน้าจัดการอุปกรณ์จอแสดงผล (Devices)



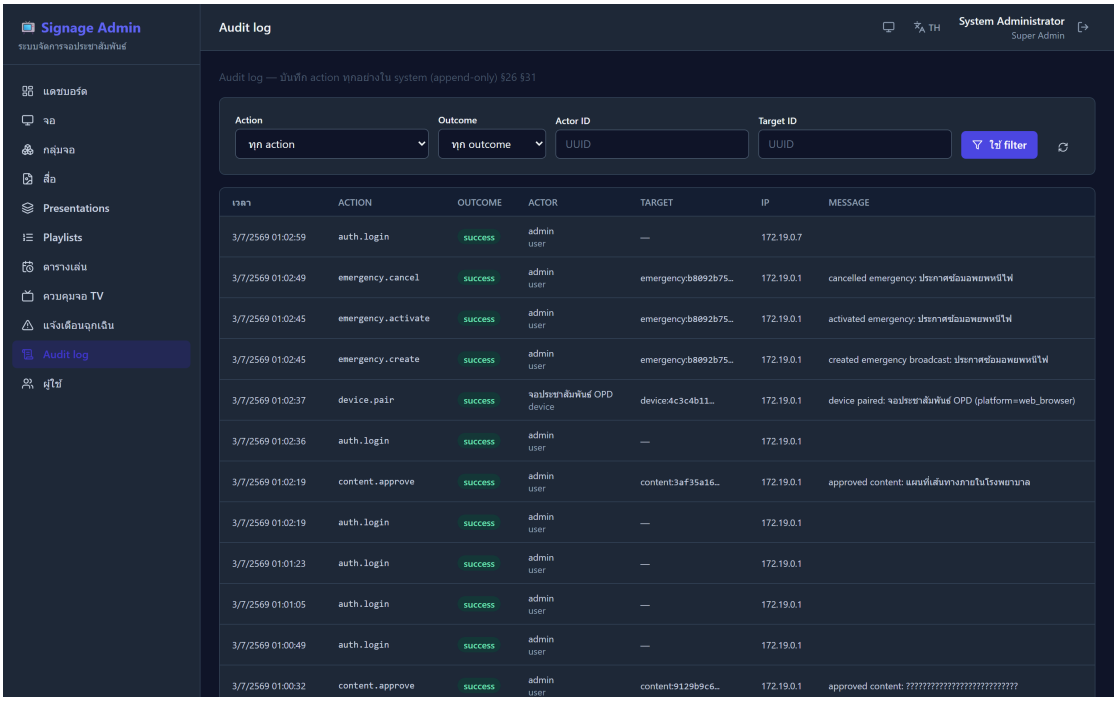
รูปที่ 4.8 หน้าควบคุมจอโทรทัศน์ (TV Control)

หน้าประกาศฉุกเฉินใช้สร้างและสั่งประกาศแทรกทุกตารางเวลา พร้อมแสดงประวัติการประกาศย้อนหลังและสถานะของแต่ละรายการ (รูปที่ 4.9) และระบบบันทึกการกระทำทุกอย่างลง

audit log แบบ append-only พร้อมตัวกรองตามประเภทการกระทำ ผลลัพธ์ และผู้กระทำ สำหรับการตรวจสอบย้อนหลัง (รูปที่ 4.10)



รูปที่ 4.9 หน้าจัดการประกาศฉุกเฉิน (Emergency)

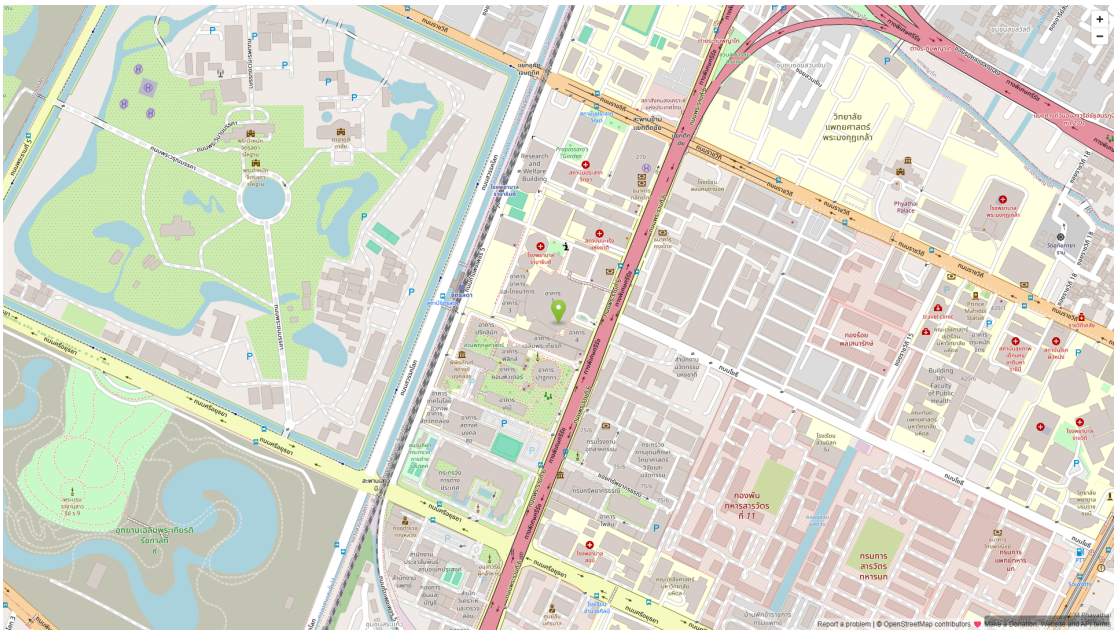


รูปที่ 4.10 หน้าบันทึกการใช้งานระบบ (Audit log)

4.4 ส่วนแสดงผล (Player)

ส่วนแสดงผลทำหน้าที่ดึง manifest จาก backend แล้วแสดงเนื้อหาตามผังและตารางที่กำหนด รองรับการแสดงหลายโซนพร้อมกัน การวนเล่นตามลำดับ และการแสดง overlay เมื่อมีประกาศฉุกเฉิน รวมถึงการ cache เนื้อหาเพื่อให้เล่นต่อได้แม้เครือข่ายขัดข้อง

รูปที่ 4.11 แสดงตัวอย่างจอแสดงผลจริงผ่าน web player หลังจับคู่อุปกรณ์กับระบบแล้ว โดยแสดงเนื้อหาเว็บ (แผนที่เส้นทางภายในโรงพยาบาล) เต็มจอตามตารางเวลาที่กำหนด และเมื่อผู้ดูแลสั่งประกาศฉุกเฉิน จอทุกจอที่เป็นเป้าหมายจะแสดง overlay ข้อความแจ้งเตือนแทรกทับเนื้อหาปกติทันที และกลับสู่การแสดงผลเดิมโดยอัตโนมัติเมื่อประกาศถูกยกเลิกหรือหมดอายุ (รูปที่ 4.12)



รูปที่ 4.11 ตัวอย่างการแสดงผลบนจอผ่าน Web Player



รูปที่ 4.12 การแสดง overlay ประกาศฉุกเฉินบนจอแสดงผล

บทที่ 5
สรุปผลและข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

โครงงานนี้ได้พัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล (Digital Signage Management System) จนสามารถใช้งานได้ครบวงจร ตั้งแต่การจัดการเนื้อหา การจัดผังการแสดงผล การกำหนดตารางเวลา การควบคุมจอโทรทัศน์และจอ LED ระยะไกล ไปจนถึงการประกาศเหตุฉุกเฉินและการเฝ้าระวังสถานะอุปกรณ์ ระบบทำงานบนสถาปัตยกรรม 6 ส่วนในรูปแบบคอนเทนเนอร์ มีส่วนจัดการสำหรับผู้ดูแล (Admin UI) และส่วนแสดงผล (Player) ทั้งบนเว็บและ Android โดยส่วนแสดงผลบนเว็บได้รับการทดสอบการใช้งานจริงตั้งแต่การจับคู่อุปกรณ์จนถึงการแสดงเนื้อหาและประกาศฉุกเฉินเต็มจอแล้ว ซึ่งบรรลุวัตถุประสงค์ทั้งหมดที่ตั้งไว้ในบทที่ 1

ในด้านคุณภาพ ระบบผ่านการทดสอบอัตโนมัติครบทั้ง 142 กรณี ครอบคลุมส่วนสำคัญของระบบ ผ่านการทดสอบแบบ End-to-End ครบทั้ง 10 ขั้นตอนบนฐานข้อมูลจริง และผ่านการทดสอบการเข้ารหัสคำสั่งควบคุมจอโทรทัศน์ครบทั้ง 47 รูปแบบด้วยตัวจำลอง อีกทั้งมีระบบ CI ตรวจสอบความถูกต้องและความปลอดภัยทุกครั้งที่แก้ไขโค้ด

5.2 ปัญหาที่พบและแนวทางแก้ไข

ระหว่างการพัฒนาพบปัญหาสำคัญและได้ดำเนินการแก้ไข สรุปได้ดังตารางที่ 5.1

ตารางที่ 5.1 ปัญหาที่พบและแนวทางแก้ไข

ปัญหาที่พบ	**แนวทางแก้ไข**
เครือข่ายภายในอาจขัดข้องชั่วคราว ทำให้จอหยุดแสดงผล	ออกแบบให้ player ดึง manifest แล้ว cache เนื้อหาไว้เล่นต่อได้แบบ offline-friendly
ทีวีมีหลายยี่ห้อ คำสั่งควบคุมต่างกัน	ออกแบบชั้นควบคุมแบบ enum protocol รองรับหลายยี่ห้อผ่าน HDMI-CEC และ RS-232
ความเสี่ยงด้านความปลอดภัยจากไฟล์อัปโหลดและ URL ภายนอก	เพิ่มการสแกนไวรัสด้วย ClamAV และการป้องกัน SSRF ในการตรวจสอบ URL
ต้องควบคุมสิทธิ์การเข้าถึงและตรวจสอบย้อนหลังได้	ใช้ระบบ RBAC 7 บทบาท ร่วมกับกระบวนการอนุมัติและการบันทึก audit log

5.3 ข้อเสนอแนะและการพัฒนาต่อยอด

แนวทางการพัฒนาต่อยอดในอนาคต (Phase 7) ได้แก่

1. ทดสอบการควบคุมกับจอโทรทัศน์และจอ LED จริงหลายยี่ห้อ เพื่อยืนยันผลที่ได้จากตัวจำลองในหัวข้อ 3.5.3 รวมถึงติดตั้งส่วนแสดงผลบน Android TV และ Raspberry Pi จริง
2. รองรับ HDMI-CEC แบบ native บนอุปกรณ์แสดงผลให้สมบูรณ์ยิ่งขึ้น
3. เสริมความปลอดภัยของการยืนยันตัวตนด้วย httpOnly cookie
4. รองรับหลายภาษา (i18n) ในส่วนจัดการ
5. ขยายระบบ CI/CD ให้ครอบคลุมการ deploy อัตโนมัติ
6. เพิ่มตารางข้อมูลแบบ time-series สำหรับ telemetry เพื่อวิเคราะห์แนวโน้มของอุปกรณ์

5.4 สิ่งที่ได้เรียนรู้จากการฝึกงาน

การปฏิบัติงานครั้งนี้ทำให้ผู้จัดทำได้เรียนรู้การพัฒนาระบบจากความต้องการใช้งานจริงอย่างครบวงจร ตั้งแต่การรวบรวมความต้องการจากผู้ใช้งาน การออกแบบสถาปัตยกรรมและฐานข้อมูล ไปจนถึงการพัฒนา ทดสอบ และติดตั้งระบบ ได้ฝึกการทำงานร่วมกันเป็นทีมโดยแบ่งความรับผิดชอบระหว่างส่วนเซิร์ฟเวอร์ (Backend) และส่วนติดต่อผู้ใช้ (Frontend) ซึ่งต้องตกลงรูปแบบการเชื่อมต่อ (API) ร่วมกันให้ชัดเจนก่อนพัฒนาแต่ละฝั่ง

ในด้านเทคนิค ผู้จัดทำได้ประสบการณ์การทำงานกับเทคโนโลยีหลายส่วนประกอบกัน ทั้งการพัฒนา API แบบ asynchronous การออกแบบระบบสิทธิ์และความปลอดภัย การเข้ารหัสคำสั่งระดับไบต์ตามโปรโตคอล HDMI-CEC และ RS-232 ตลอดจนการสร้างตัวจำลอง (emulator) ขึ้นมาตรวจสอบความถูกต้องของคำสั่งเมื่อยังไม่มีอุปกรณ์จริงให้ทดสอบ ซึ่งเป็นบทเรียนสำคัญว่างานที่ต้องเชื่อมต่อกับฮาร์ดแวร์ยังสามารถทดสอบได้อย่างเป็นระบบ นอกจากนี้ยังได้ใช้เครื่องมือมาตรฐานอุตสาหกรรม ได้แก่ Git ในการจัดการเวอร์ชันโค้ด Docker ในการติดตั้งระบบ และการเขียนชุดทดสอบอัตโนมัติ ซึ่งช่วยให้ตรวจพบข้อผิดพลาดได้ตั้งแต่ระหว่างการพัฒนา ประสบการณ์เหล่านี้สามารถนำไปประยุกต์ใช้ในการทำงานจริงต่อไปได้

บรรณานุกรม

Docker Inc. (ม.ป.ป.). *Docker Documentation*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://docs.docker.com

HDMI Licensing Administrator, Inc. (ม.ป.ป.). *HDMI Specification: Consumer Electronics Control (CEC)*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://www.hdmi.org

Jones, M., Bradley, J., & Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://www.rfc-editor.org/rfc/rfc7519

MinIO, Inc. (ม.ป.ป.). *MinIO Object Storage Documentation*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://min.io/docs

PostgreSQL Global Development Group. (ม.ป.ป.). *PostgreSQL 16 Documentation*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://www.postgresql.org/docs/16/

pytest-dev team. (ม.ป.ป.). *pytest Documentation*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://docs.pytest.org

Python Software Foundation. (ม.ป.ป.). *Python 3.12 Documentation*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://docs.python.org/3.12/

Ramírez, S. (ม.ป.ป.). *FastAPI Documentation*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://fastapi.tiangolo.com

Redis Ltd. (ม.ป.ป.). *Redis Documentation*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://redis.io/docs

SQLAlchemy authors. (ม.ป.ป.). *SQLAlchemy 2.0 Documentation*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://docs.sqlalchemy.org/en/20/

Tailwind Labs Inc. (ม.ป.ป.). *Tailwind CSS Documentation*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://tailwindcss.com/docs

Telecommunications Industry Association. (1997). *TIA/EIA-232-F: Interface between data terminal equipment and data circuit-terminating equipment employing serial binary data*

interchange. Arlington, VA: TIA.

Vercel Inc. (ม.ป.ป.). *Next.js Documentation*. สืบค้นเมื่อ 2 กรกฎาคม 2568, จาก https://nextjs.org/docs